

Algoritmer

Generics



Agenda

- Hvad er generics
- Performance
- Øvelse – generisk swap kode
- Type safety
 - Genbrug på binært niveau
- Linked list uden Generics
- Øvelse – linkedlist uden generics
- Generisk kode
 - Generiske klasser, Navngivning, Default Values, Constraints, Nedarving, Static Members, Generic Interface
- Linked list med genericsSortering via indbygget funktionalitet
- Generisk liste.
- Øvelse – linkedlist med generics
- Øvelse sortering/generisk liste





Hvad er generics?



Hvad er generics?

- Introduceret i C# 2.0 og understøttet fra .NET 2.0.
- Gør det muligt at lave klasser og metoder, der er uafhængige af en specifik datatype (f.eks. int, string).
- Typisk angives typen med T, men andre navne kan bruges.
- Samme metode eller klasse kan bruges til flere forskellige typer, hvilket reducerer redundant kode.
 - Eksempel: Én generisk metode kan erstatte separate metoder til f.eks. int og float.
- Eksempel på en generisk klasse:

```
0 references
public class Box<T>
{
    0 references
    public T Value { get; set; }
}
```



Hvad er generics?

- Eks. Hvis man har et program, som skal kunne ændre værdien på en variabel

```
string word = "Hello";
```

```
int number = 1;
```



Hvad er generics?

- Uden generics, vil det kræve følgende metoder:

```
private static string ChangeValue(string value)
{
    return value;
}
```

```
private static int ChangeValue(int value)
{
    return value;
}
```

- Kaldet vil se ud på følgende måde:

```
//Defines the default values
string word = "Hello";
```

```
int number = 1;
```

```
//Changes the values
word = ChangeValue("Hello World");
```

```
number = ChangeValue(13);
```



Hvad er generics?

*Kan man ikke bare bruge
Object klassen for at reducere
mængden af kode?*



- Man kan godt bruge Object, men det er ikke typestærkt og kræver typekonverteringer såsom boxing og unboxing!
- Med en generisk klasse bliver typen erstattet med den korrekte ved kompilering, hvilket sikrer typesikkerhed, bedre performance og færre fejl.



Performance



Performance

- En stor fordel ved generics er bedre performance
 - Man undgår boxing og unboxing når referencetyper laves til værdityper og omvendt



Performance: Boxing & Unboxing

- .NET konverterer automatisk værdityper til referencetyper
 - Værdityper gemmes på stack (Structs, ints etc.)
 - Referencetyper gemmes på heap (Classes, string)
- Værdityper kan bruges alle steder et object efterspørges
 - En int kan f.eks. gemmes som et object
 - *Dette er boxing*
 - Unboxing sker når vi går den anden vej
 - *Fra reference til værditype*
 - Her bruges der et cast

```
int myInt = 30;
```

```
Object tmp = myInt; //This is boxing
```

```
myInt = (int)tmp; //This is unboxing
```



Performance: Boxing & Unboxing

- Hvorfor er dette vigtigt?
 - Boxing/unboxing koster performance, fordi værdier kopieres og flyttes mellem stack* og heap**. Generics undgår dette, fordi typen forbliver intakt.

*Stack: Hurtig hukommelse, bruges til lokale variabler og værdityper. Automatisk administreret (LIFO – Last In, First Out).

**Heap: Langsommere, men fleksibel hukommelse, bruges til objekter og referencetyper. Skal ryddes manuelt (Garbage Collection).



Tilbage til vores ChangeValue eksempel

- En generisk metode angives på følgende måde

```
[Output data type] [metodenavn]<T>(T input)
{
}
```



Hvad er generics?

- Uden generics →

```
private static string ChangeValue(string value)
{
    return value;
}

private static int ChangeValue(int value)
{
    return value;
}
```

```
//Defines the default values
string word = "Hello";

int number = 1;

//Changes the values
word = ChangeValue("Hello World");

number = ChangeValue(13);
```

- Ved brug af generics, kan man nøjes med følgende metode:

```
private static T ChangeValue<T>(T value)
{
    return value;
}
```



Hvad er generics?

- Kaldet vil se ud på følgende måde:
 - I kender allerede strukturen fra lister
 - Typen kan angives manuelt, men bestemmes automatisk, hvis det ikke er nødvendigt:

```
firstString = ChangeValue<string>("Hello World");
```

```
firstFloat = ChangeValue<float>(400);
```

```
firstint = ChangeValue(123); // T= int bestemmes automatisk
```



Øvelse

1. Implementer en generisk version af Swap metoden og test at den virker med mindst 2 forskellige datatyper.

```
private static void Swap(ref object first, ref object second)
{
    object tmp = first;
    first = second;
    second = tmp;
}
```



ArrayList vs List<T>

ArrayList fra System.Collections.ArrayList
VS

List<T> fra System.Collections.Generic



ArrayList vs List<T>

- ArrayList eksempel
 - Boxing og unboxing sker
 - Høj performance impact

```
int ArrayList.Add(object value)  
Adds an object to the end of the System.Collections.ArrayList.  
  
Exceptions:  
System.NotSupportedException
```

```
ArrayList list = new ArrayList();  
  
list.Add(13); //Boxing - Convert a value type to a reference type  
  
int i = (int)list[0]; //Unboxing - Convert a reference type to a value type  
  
foreach (int i2 in list)  
{  
    Console.WriteLine(i2); //Unboxing  
}
```



ArrayList vs List<T>

- List<T> eksempel
 - Definerer den korrekte type i stedet for at bruge object
 - Boxing og unboxing er ikke nødvendig

```
List<int> list = new List<int>();  
  
list.Add(13); //No boxing - value types are stored in the List<int>  
  
int i = list[0]; //No unboxing, no cast needed  
  
foreach (int i2 in list)  
{  
    Console.WriteLine(i2); //No unboxing  
}
```





Type safety



Type Safety

- ArrayList er ikke typestærk

```
ArrayList aList = new ArrayList();  
  
aList.Add(1);  
  
aList.Add("Hello World");  
  
foreach (int item in aList) //This will throw an exception  
{  
    Console.WriteLine(item);  
}
```

- List<T> er typestærk

```
List<int> list = new List<int>();  
  
list.Add(13);  
  
list.Add("Hello World"); //Compile time error
```



Genbrug på binært niveau

- En generisk klasse er defineret 1 gang
 - Kan instantieres som flere typer
 - Kompileres til den specifikke type med (JIT)
 - Giver mindre kode

```
List<int> intList = new List<int>();  
intList.Add(30);
```

```
List<string> strList = new List<string>();  
strList.Add("Hello World");
```



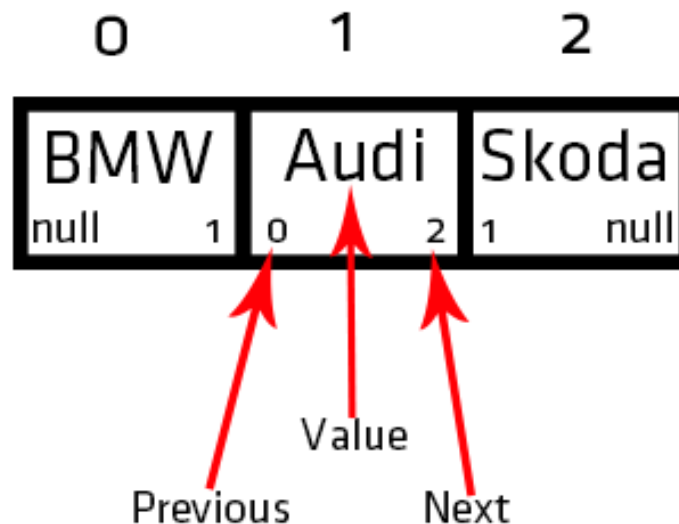
LinkedList uden generics



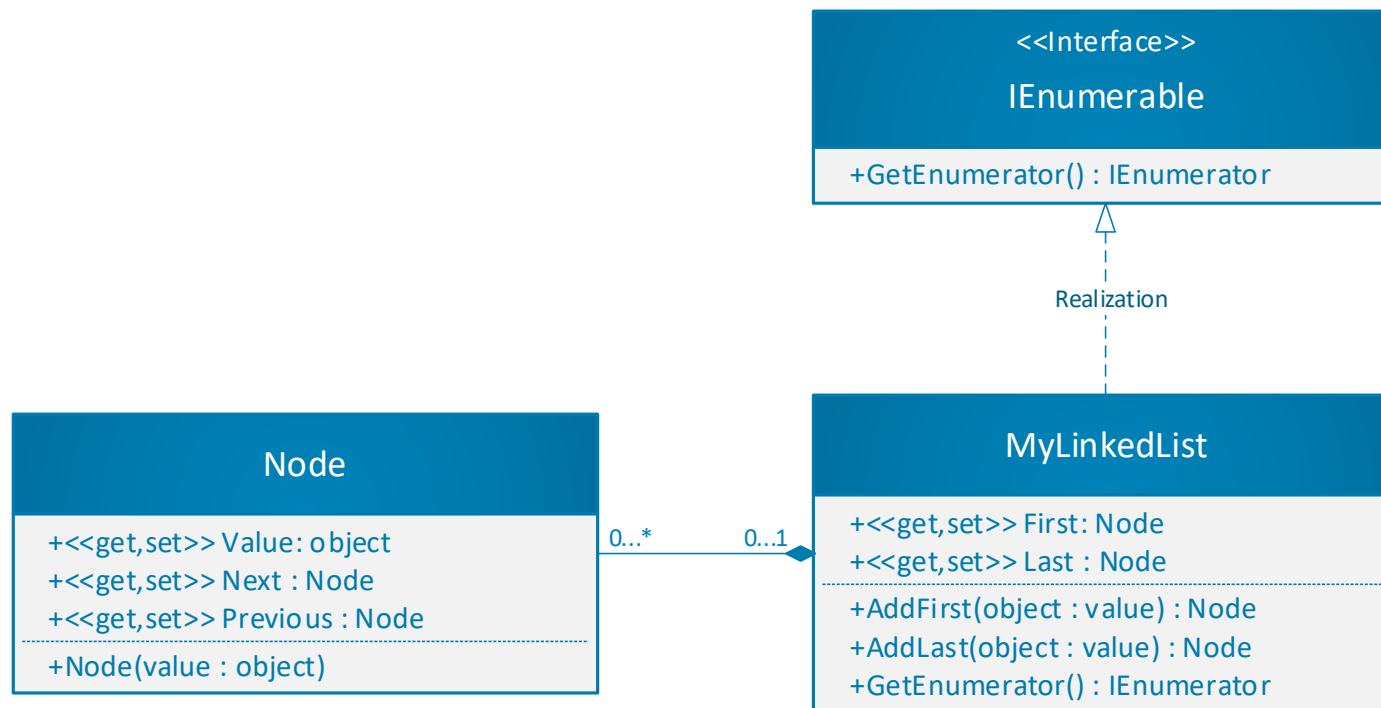
LinkedList uden Generics

Simpelt linked list eksempel, uden brug af Generics:

- En LinkedList består af noder, hvor hver node peger på den næste og forrige node.
- I C# er **LinkedList** en **dobbelt-linket liste** (*doubly linked list*), hvilket betyder, at man kan traversere listen i begge retninger.



LinkedList uden Generics



LinkedList uden Generics

- Node
 - Value: Værdien, som noden indeholder
 - Previous: Den foregående node
 - Next: Den næste node

Node
+<<get,set>> Value: object
+<<get,set>> Next : Node
+<<get,set>> Previous : Node
+Node(value : object)



LinkedList uden Generics

- Herefter kan listen laves
 - Vi vil gerne kunne iterere med en foreach over listen
 - Derfor skal **IEnumerable** interfacet implementeres
 - *Bruges til iteration over en non-generic collection*

```
class MyLinkedList : IEnumerable
```

<<Interface>>

IEnumerable

+GetEnumerator() : IEnumerator



LinkedList uden Generics

- MyLinkedList, holder styr på listestrukturen
 - First: Det første element på listen
 - Last: Det sidste element på listen
 - Add First: Tilføjer et element først på listen
 - Add Last: Tilføjer et element sidst på listen
 - GetEnumerator: Bruges til at iterere over listen

MyLinkedList

+<<get,set>> First: Node

+<<get,set>> Last : Node

+AddFirst(object : value) : Node

+AddLast(object : value) : Node

+GetEnumerator() : IEnumerator



LinkedList uden Generics – AddFirst

- Metoden AddFirst(object value) indsætter en ny node i starten af listen
 - Node **n** = new Node(value);
- Hvis listen er tom (First == null), bliver **n** både First og Last.
- Hvis listen ikke er tom:
 - Den nuværende First node får **n** som Previous.
 - **n** peger på den gamle First node (**n**.Next = First).
 - First bliver opdateret til **n**.

```
if (First == null)
{
    First = Last = n;
}
else
{
    First.Previous = n;
    n.Next = First;
    First = n;
}
```



LinkedList uden Generics - AddLast

- Metoden AddLast(object value) indsætter en ny node i slutningen af listen.
 - Node **n** = new Node(value);
- Hvis listen er tom (First == null), bliver n både First og Last.
- Ellers opdateres referencer:

```
n.Previous = Last;  
Last.Next = n;  
Last = n;
```



LinkedList uden Generics

- GetEnumerator() gør foreach og LINQ muligt med en LinkedList
- Hver iteration returnerer ét element ad gangen. Eksempel:

```
public IEnumerator GetEnumerator()  
{  
    Node current = First;  
  
    while (current != null)  
    {  
        yield return current.Value;  
        current = current.Next;  
    }  
}
```

Hvis GetEnumerator() ikke er implementeret får man følgende fejl ved brug af foreach:

 1 'Generics.MyLinkedList' does not implement interface member 'System.Collections.IEnumerable.GetEnumerator()'

LinkedList uden Generics

- Yield return returnerer hvert element et efter et
 - Når der returneres med yield return i en iterator metode bevares positionen i koden
 - Eksekveringen fortsætter fra den position næste gang iterator metoden kaldes

```
yield return current.Value;
```



LinkedList uden Generics

- Eksempel på brug

```
MyLinkedList linked = new MyLinkedList();
```

```
linked.AddLast(3);  
linked.AddLast(8);  
linked.AddLast(7);  
linked.AddFirst(13);
```

```
foreach (int item in linked)  
{  
    Console.WriteLine(item);  
}
```

13

3

8

7



Øvelse

2. Lav en LinkedList, som **ikke** er generisk

- Tilføj metoderne AddFirst() og AddLast()
- Brug en foreach løkke til at skrive listens indhold ud
- Der skal ikke gøres brug af .NET's indbyggede **arrays** eller **lister** til at løse opgaven





Generisk kode



Generiske klasser

- Strukturen af en generisk klasse ser ud på følgende måde:

```
public class[klassenavn]<[TInputs]>
{
//Klassens kode
}
```



Generiske klasser

- Simpelt generisk eksempel

```
class ValueClass<T>
{
    private T value;

    public ValueClass(T value)
    {
        this.value = value;
    }
}
```

- T bliver erstattet af den datatype klassen instantieres med. I dette tilfælde vil det være int
 - Dette kender vi fra generiske liste

```
ValueClass<int> myValue = new ValueClass<int>(13);
```



Navngivning

- Navne på generiske typer bør have T prefix
 - F.eks. THighScoreEntry
- Hvis den generiske type kan erstattes af alle klasser er det ok bare at bruge T
 - F.eks. List<T>
- Hvis der bruges flere generiske typer bør man bruge beskrivende navne
 - F.eks. SortedList<TKey, TValue>
 - Dictionary<Tkey,Tvalue>



Default Values

- En generisk type kan være både værditype og referencetype
- **Værdityper** (int, float) kan **ikke** være **null**.
- **Referencetyper** (string, object) kan være **null**.
- Brug `default(T)` i stedet for **null**, når typen er ukendt.

```
public T GetValue()
{
    T value = default(T);

    try
    {
        //Kode som kan fejle (hent en highscore)
    }
    catch (Exception)
    {
    }

    return value;
}
```



Default Values

- Default(T) i Generiske Metoder:
- For værdityper returnerer default(T) standardværdien (fx 0 for int, false for bool)
- For referencetyper returnerer default(T) "null"
- Default(T) er nyttigt til at initialisere variabler og oprette nye objekter

```
public T GetValue()
{
    T value = default(T);

    try
    {
        //Kode som kan fejle (hent en highscore)
    }
    catch (Exception)
    {
    }

    return value;
}
```



Constraints

- Man kan begrænse, hvilke typer der tillades i en generisk klasse/metode med where:

```
public class MyClass<T> where T : MyOtherClass
{
    // T skal være MyOtherClass eller nedarve fra den
}
```

Eksempel:

```
public class PetStore<T> where T : Animal
{
    // T skal være animal eller nedarve fra den
}
```

```
class Animal { }
class Dog : Animal { }
```

PetStore<Dog> obj = new PetStore<Dog>(); // dette er OK

PetStore<int> obj2 = new PetStore<int>(); // ✗ Fejl, int er ikke Animal



Constraints

- Eks: Nogle dyr kan flyve

```
interface IFlyable      class Animal
{
    void Fly();          {
}
```

- Vi har følgende dyr i vores projekt:

```
class Duck : Animal, IFlyable
{
    public void Fly()
    {
        Console.WriteLine("I'm flying");
    }
}
```

```
class Tiger : Animal
{
}
```



Constraints

- Vi har en cage klasse, som kan indeholde dyr

```
class Cage<TAnimal>
{
    private List<TAnimal> animals = new List<TAnimal>();

    public void AddAnimal(TAnimal animal)
    {
        animals.Add(animal);
    }

    public void Test()
    {
        foreach (TAnimal animal in animals)
        {
            (animal as IFlyable).Fly();
        }
    }
}
```



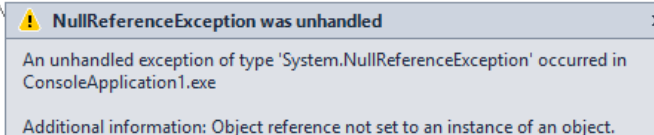
Constraints

- Følgende kode vil give en exception
 - Fordi Tiger ikke kan flyve

```
Cage<Animal> cage = new Cage<Animal>();
```

```
cage.AddAnimal(new Duck());  
cage.AddAnimal(new Tiger());  
cage.Test();
```

```
public void Test()  
{  
    foreach (T animal in animals)  
    {  
        (animal as IFlyable).Fly();  
    }  
}
```



NullReferenceException was unhandled

An unhandled exception of type 'System.NullReferenceException' occurred in ConsoleApplication1.exe

Additional information: Object reference not set to an instance of an object.



Constraints

- Følgende constraint kan bruges til at undgå fejlen:

```
class Cage<TAnimal> where TAnimal : IFlyable
```

- Cage virker nu kun med klasser, som implementerer IFlyable
- Metoden vil nu se ud på følgende måde
 - Vi slipper også for at caste til IFlyable

```
public void Test()  
{  
    foreach (TAnimal animal in animals)  
    {  
        animal.Fly();  
    }  
}
```



Constraints

- Kode til at instantiere skal nu ændres til følgende
 - Det er ikke længere muligt at putte en tiger i cage, fordi den ikke er IFlyable

```
Cage<IFlyable> cage = new Cage<IFlyable>();  
cage.AddAnimal(new Duck());  
cage.Test();
```



Nedarving

- En generisk klasse kan nedarve fra en generisk klasse (kan også nedarve fra non generic)
 - Den generiske type kan gentages så den nedarvede funktionalitet bruger samme datatype

```
class BaseGeneric<T> : SuperGeneric<T>
{
}

```

```
class SuperGeneric<T>
{
}

```

- Datatypen kan specificeres, så den nedarvede funktionalitet kun kan arbejde med den specifikke type

```
class BaseGeneric<T> : SuperGeneric<string>
{
}

```



Static Members

- Statiske medlemmer deles mellem forskellige instantieringer af klassen baseret på deres specifikke type

```
class BaseGeneric<T>
{
    public static int x;
}
```

```
BaseGeneric<int>.x = 30;
BaseGeneric<string>.x = 40;
```

```
Console.WriteLine(BaseGeneric<int>.x);
Console.WriteLine(BaseGeneric<string>.x);
```

```
30
40
Press any key to continue . . . _
```



Generic interfaces

- **Definition:** Generiske interfaces tillader fleksibel implementering af interfaces, hvor typen ikke er defineret på forhånd.
- **Reducerer typecasts:** Ved at bruge generiske interfaces kan vi undgå behovet for typecasts, hvilket gør koden mere læsbar og mindre tilbøjelig til fejl
- **Anvendelse i samlinger:** Generiske interfaces er særligt nyttige i forbindelse med samlinger, da de tillader os at arbejde med forskellige datatyper uden tab af typeinformation
- **Eksempler:** Nogle almindelige generiske interfaces inkluderer `Comparable<T>` og `ICollection<T>`



{ Generic interfaces

- **Comparable<T>:**
- **Formål:** Dette interface tillader objekter af typen T at sammenligne sig med andre objekter af samme type.
- **Metode:** Det definerer en enkelt metode kaldet CompareTo, som objekter, der implementerer interfacet, skal implementere. Denne metode sammenligner det aktuelle objekt med et andet objekt af samme type og returnerer en værdi, der angiver deres relative orden.
- **Anvendelse:** Det bruges ofte til sortering og sammenligning af objekter i samlinger eller andre situationer, hvor sammenligning er nødvendig.



{ Generic interfaces

- **ICollection<T>:**
- **Formål:** Dette interface repræsenterer en generisk samling af objekter af typen T.
- **Metoder og egenskaber:** Det definerer metoder og egenskaber til at tilføje, fjerne og manipulere elementer i samlingen, samt metoder til at undersøge samlingens størrelse og til at kopiere elementerne til et array.
- **Anvendelse:** Det bruges til at arbejde med samlinger af objekter af en bestemt type. Implementeringer af dette interface findes i forskellige samlingstyper såsom lister og dictionaries, der tilbyder forskellige funktioner til datastyring og -håndtering



{ Generic interfaces

- Ældre versioner:
 - Tidligere versioner af .NET, såsom .NET 1.0, havde ikke generiske versioner af interfaces som `IComparable`. Dette førte ofte til kompleksitet og tab af ydeevne, da typecasts var nødvendige



Generic interfaces

Ikke generisk

- Her skal der castes

```
class Person : IComparable
{
    private string lastName;

    public string LastName...

    /// <summary>
    /// Implementering af IComparable interfaces
    /// </summary>
    public int CompareTo(object obj)
    {
        //Caster objektet til en person
        Person other = obj as Person;

        //Returnerer sammenligningen
        return this.lastName.CompareTo(other.lastName);
    }
}
```

Generisk

- Her skal ikke castes

```
class Person : IComparable<Person>
{
    private string lastName;

    public string LastName...

    /// <summary>
    /// Implementering af IComparable<T>
    /// </summary>
    public int CompareTo(Person other)
    {
        //Returnerer sammenligningen
        return this.lastName.CompareTo(other.lastName);
    }
}
```



Sortering af liste

- Når en liste sorterer objekter skal den vide hvordan objekterne skal vurderes
 - Listen har nemlig ingen mulighed for at vide hvilke kriterier den skal bruge for at sortere følgende klasse:
 - Skal scoren eller navnet vægtes højest, er 0 en bedre score end 1 eller er 1 en bedre score en 0

HighScore
-score : int -name : string



Sortering af liste

- Til at håndtere sorteringen gør listen brug af IComparable interfacet.
 - Dvs. at den leder efter en implementering af dette interface i klassen, når der kaldes .Sort(); på en liste med objekter.
 - Der findes både en generisk og en ikke generisk version
 - Obj og other er det objekt der sammenlignes med

IComparable

+CompareTo(obj : object) : int

IComparable<T>

+CompareTo(other : T) : int



Sortering af liste

- Bemærk at begge versioner returnerer en int.
- Det er fordi listen skal bruge en int værdi til at vide hvor elementet skal placeres.
 - Listen sammenligner 2 elementer ad gangen.
 - Hvis 0 returneres, er værdierne ens: A & B er ens
 - Hvis 1 returneres er det nuværende element højere end det andet element: A vægtes højere end B
 - Hvis -1 returneres er det nuværende element lavere end det andet element: A vægtes lavere end B
 - Dvs. at I skal definere hvornår der skal returneres -1, 0 og 1.

IComparable

+CompareTo(obj : object) : int

IComparable<T>

+CompareTo(other : T) : int

Øvelse

fortsættelse på 022

3. Gør jeres LinkedList generisk, så I undgår boxing og unboxing

- Husk at implementere den generiske version af IEnumerable:
I MyLinkedList klassen

```
IEnumerable<T>
```

```
MyLinkedList<int> linked = new MyLinkedList<int>();
```

```
linked.AddLast(3);  
linked.AddLast(8);  
linked.AddLast(7);  
linked.AddFirst(13);
```

```
foreach (int item in linked)  
{  
    Console.WriteLine(item);  
}
```



Øvelse

øvelse 023

- 4. Lav en highscore klasse, som indeholder
 - Navn og score
 - Tilføj 10 highscores til en liste (en normal generisk liste fra .NET).
 - Sorter listen via Sort(), bemærk at det ikke er muligt
 - Implementer den generiske version af IComparable
 - Sorter først på scoren herefter på navnet.
 - Hint. Brug CompareTo på name variabelen til at sammenligne to navne.
 - Returnere allerede hhv -1,1 og 0 😊



Øvelse

øvelse 024

- 5. Implementer din egen version af en generisk liste.
 - Der stilles følgende krav til opgaven.
 - Du skal kunne løbe listen igennem med en foreach løkke.
 - Listen skal have følgende metoder:
 - Add(T element) Tilføjer et element til listen.
 - Remove(T element) Fjerne et element fra listen.
 - Clear(): Fjerner alt indholdet på listen.
 - Sort(): Sorterer listen
 - Du må ikke benytte dig af de indbyggede collections i .NET
 - Ingen stacks, queues, dicts, etc...
 - Du må gerne benytte dig af arrays.
 - Array har en ".Sort" 😊

